

30.5 A 16nm 72kb 120.5TFLOPS/W Versatile-Format Dual-Representation Gain-Cell CIM Macro for General Purpose AI Tasks

Jen-Chun Tien^{*1}, Win-San Khwa^{*2}, Le-Jung Hsieh¹, Tsung-Han Lou¹, Jyun-Cheng Bai¹, Yu-Sheng Kao¹, Ting-Hao Hsu¹, Mai Tseng¹, Hung-Hsi Hsu¹, Yao-Kai Yeh¹, De-Qi You¹, Ashwin Sanjay Lele³, Brian Crafton³, Bo Zhang³, Ping-Sheng Wu², Ya-Tang Yang¹, Chung-Chuan Lo¹, Ren-Shuo Liu¹, Chih-Cheng Hsieh¹, Kea-Tiong Tang¹, Meng-Fan Chang^{1,2}

¹National Tsing Hua University, Hsinchu, Taiwan, ²TSMC Corporate Research, Hsinchu, Taiwan, ³TSMC Corporate Research, San Jose, CA

^{*}Equally Credited Authors (ECAs)

Abstract

Diverse AI workloads require flexible data representations and numerical formats. In this work, we present a reconfigurable 2's-complement and sign-magnitude scheme integrated within a versatile-format CIM macro supporting MX, LNS, FP, and INT for MAC operations.

Implemented in a 16nm 72kb gain-cell array, the macro achieves record energy efficiency (120.5TFLOPS/W) and throughput density (3.18 TOPS/mm²) in MXINT8 mode.

Diverse AI workloads require flexible compute-in-memory (CIM) architectures that support multiple numerical formats (e.g., floating-point (FP) and integer (INT)) and data representations (e.g., 2's complement (2C) and sign magnitude (SM)) to balance accuracy, area, and energy efficiency. Commercial accelerators have begun adopting micro-scaling (MX) to train low-precision large language models (LLMs) [1,4], while the logarithmic number system (LNS) has shown promise for spectral-based AI kernels (e.g., Fourier neural operator) [24-29] due to its multiplication efficiency. However, most existing CIM architectures are tailored to a specific representation or format [5-22], limiting adaptability across diverse AI tasks and deployment scenarios. Figure 30.5.1 illustrates the key challenges in developing CIM for general-purpose AI workloads: (1) 2C versus SM area-power trade-off: 2C representation suffers from high toggle rates due to low bit-wise sparsity near negative zero, whereas SM increases hardware area overhead. (2) Fixed MX block size: Using a single MX block-size (k) constrains the energy-accuracy trade-off as it cannot adapt to different input distributions. (3) LNS area overhead: Supporting LNS in CIM requires replacing MAC operations with additions and look-up table (LUT) accesses, increasing area overhead and reducing area utilization since hardware cannot be shared across formats.

To address these challenges, this work presents a versatile-format (MX-LNS-FP-INT) dual-representation (2C-SM) gain-cell CIM (VF-DR GC-CIM) for general-purpose AI tasks, featuring: (1) Compact in-situ weight (W) sparsity booster (CIWSB) with polarity-shift W control (PSWC): Transfers weight polarity to inputs (INs) to overcome the area-energy trade-off between 2C and SM and enables reconfigurable 2CSM computation in CIM macros with minimal area overhead; (2) Input distribution-aware MX quantizer (IDA-MXQ): Adaptively selects the optimal MX block-size k based on IN distributions, improving both energy efficiency and accuracy; (3) Multi-format adaptive computing cell (MFA-CC) with delta exponent subtractor (Δ ES) and LUT support: Shares hardware across LNS, FP, and INT modes. The Δ ES increases bit-wise sparsity in LNS mode, enabling high area utilization and energy efficient computation.

We fabricated a 16nm 72kb VF-DR GC-CIM supporting versatile-format MAC operations. Leveraging gain-cell memory for high storage density, the macro achieves energy efficiency of 120.5TFLOPS/W with area efficiency of 3.18TOPS/mm² in MXINT8 mode, 98.1TFLOPS/W with 3.18TOPS/mm² in LNS8 mode, 50.4TFLOPS/W with 1.59TOPS/mm² in BF16 mode, and 138.5TOPS/W with 3.18TOPS/mm² in INT8 mode.

Figure 30.5.1 illustrates the structure of the proposed VF-DR GC-CIM, comprising four tile-based matrix-matrix CIM clusters. Each cluster contains an IDA-MXQ and four CIM banks. Each CIM bank includes 32 sub-banks, an alignment and adder tree, and a FP converter. Each sub-bank consists of a gain-cell array (8 rows $\times$ 18 columns), an IN processor, a CIWSB, and four MFA-CC units. Each CIM bank provides four output channels, broadcasting the same weight to four different inputs across MFA-CC units. The outputs from the four CIM banks are combined to produce the final matrix-matrix result.

Figure 30.5.2 shows the proposed CIWSB. By shifting the polarity of Ws to INs and retaining Ws in positive 2C (P2C) representation, the average MFA-CC power consumption is reduced by 31%. This reduction arises because the MUX-based multiplier [12] in MFA-CC is more sensitive to W sparsity than IN sparsity. Conventional digital multipliers also benefit from this approach, as cases where (IN, W)=(-, -) can be converted to (+, +), effectively increasing sparsity. CIWSB increases W bit-wise sparsity by 20.1%, improving power efficiency in weight-stationary CIM flows. Moreover, the positive SM representation aligns with P2C, enabling direct execution of SM computations on 2C MAC units. This eliminates the for negative adders or subtractors [23] improving area efficiency for SM MAC operations.

The CIWSB comprises 18 stationary gated units (SGUs) and two PSWCs. During computation, the SGUs retain the weights read from the gaincell array, while the PSWCs and SGUs together convert them into a P2C representation. The encoding flow operates as follows: In 2C mode, when a W is negative, the PSWC converts it to positive by locating the first "1" from the LSB and setting all CTRL signals from the MSB down to this bit at 1. This reconfigures the SGU readout path, after which the SGU inverts the readout value. Unlike the conventional invert-plus-one method [30], which relies on numerous power-hungry halfadders, PSWC achieves the same function while reducing power consumption by 91.24% and area overhead by 76.4%. In SM mode, the PSWC sets the MSB of CTRL to 1 for negative W, enabling direct computation of SM data on 2C MAC units to enhance energy

efficiency. Finally, IN processor converts INs (2C or SM) into corresponding 2C, while adjusting their polarity according to the polarity of W.

Figure 30.5.3 illustrates the proposed IDA-MXQ, which dynamically adjusts the block-size (k) for each IN data patch within a single cycle, thereby enhancing computational efficiency without compromising accuracy. Our analysis shows that the k of each MX group should be adaptively determined according to the variance of the data distribution, rather than adopting a uniform k across all neural network models [24]. For high-variance data, a smaller k preserves the data range during quantization, though at the cost of higher computational overhead. For low-variance data, a larger k improves computational efficiency by grouping more inputs into an MX block, with only a minor loss due to quantization distortion, particularly for outliers. IDA-MXQ adaptively quantizes FP32 data patches to MX-INT8/FP8/LNS8 with k ranging from 4 to 64, depending on the distribution variance. Unlike conventional approaches that combine computation across multiple MX blocks and thereby accumulate both partial results and quantization errors [6], this design completes MAC operations entirely within each MX block and outputs FP32 results directly, ensuring that MX acceleration does not significantly degrade accuracy.

The IDA-MXQ consists of two main components: (1) an input distribution-aware k detector (IDA-KD), which selects the optimal k for each patch of 64 INs, and (2) an FP to MX quantizer (FP2MX), which quantizes FP32 data into MX format to accelerate MAC operations. IDA-KD and FP2MX both complete their operations within one cycle.

IDA-KD operates in three phases. In phase 1, an exponent frequency counter (EFC) records the frequency of each input exponent (IN_E) within the high-probability region (HPR). In most neural network models, the IN_E distribution is concentrated within a narrow HPR, enabling IDA-KD to significantly reduce area and latency overhead in variance detection. For ImageNet-1K inference with MobileViT, HPR is set between -15 and 2, predetermined during training. In phase 2, a region aggregator (RA) aggregates frequencies within regions A, B, and C (F_A , F_B , F_C) and extracts the most frequent value (MFV). In Phase 3, a k finder (KF) determines the optimal k based on F_A , F_B , F_C , and MFV. In the following cycle, FP2MX quantizes the FP32 data patch to MX with the optimal k . IDA-MXQ adaptively selects the optimal k by balancing the trade-off between precision and efficiency. Moreover, by limiting detection to HPR, the IDA-MXQ significantly reduces latency and increases area efficiency compared to full-range detection methods.

Figure 30.5.4 illustrates the proposed MFA-CC. Each sub-bank integrates 4 MFA-CCs, which share the same Ws ($M_0[8:0]$, $M_1[8:0]$) broadcasted from the array, while receiving four independent inputs from IDA-MXQ for matrix-matrix MAC operations. An MFA-CC consists of 18 bit-wise reconfigurable units (BRUs), two Δ ES, and two LUTs. Each BRU can be configured as either a bit-wise full adder or a multiplier [12], enabling support for LNS, FP, and INT MAC operations within the same hardware and improving area utilization across formats. In 8b reconfigurable mode, BRU is divided into two parts to support reconfigurable MAC operations.

Unlike FP or INT multiplication, which requires costly multipliers, LNS multiplication uses simple adders followed by a LUT [27-28]. The proposed Δ ES with LUT improves energy efficiency by enhancing sparsity in LNS-based LUT results. During alignment, conventional LNS LUTs read out the entire LUT length, including truncated bits [27-28], causing unnecessary bit toggles in both the LUT and alignment circuitry. In contrast, the Δ ES generates an access mask based on the delta product exponent (ΔPD_E). The access mask bit-width (AMB) is set at $8 - \Delta PD_E$, where 8 is the LUT length. By masking out truncated bits, the Δ ES avoids unnecessary readouts without a loss of accuracy, increases LSB sparsity during alignment, and reduces alignment energy. In FP mode, the sign and leading-1 are pre-stored in memory and read out when combined with multiplication. Δ ES is reconfigured to compute the number of shifting bits (NS) of products, further improving area utilization across formats beyond LNS.

Figure 30.5.5 summarizes the performance of the proposed schemes. CIWSB improved FoM1 by 1.37 to 6.81 $\times$ compared with fixed-representation CIM designs by increasing weight bit-wise sparsity to improve energy efficiency in 2C mode, while reusing the same mechanism to increase area efficiency in SM mode. IDA-MXQ achieved a 1.29 $\times$ FoM2 gain over MX-CIM with a fixed block-size by adaptively selecting the optimal block-size according to the input data distribution. MFACC improved FoM2 by 1.49 to 1.87 $\times$ over conventional versatile-format CIM implementations, by enabling hardware sharing across

MX-LNS-FP-INT formats and providing support for LNS. Finally, VF-DR GC-CIM achieved a 1.7 to 2.3 $\times$ improvement in FoM1 across representative AI workloads, demonstrating that supporting MXLNSFPINT format provides a range of benefits in general-purpose CIM.

Figure 30.5.6 presents measured results and a comparison table of the proposed VFDR GCCIM performing versatile-format MAC operations. Shmoo plots indicated a 1.2ns access time for MXINT8-MAC operation using a 0.8V power supply. Figure 30.5.7 presents a die micrograph and chip performance summary table. Product alignment and full-output precision preserved inference accuracy, with only a <0.01% loss across all formats, when applying ResNet-20 to CIFAR-100 and MobileViT to ImageNet-1K.

Acknowledgement:

The authors would like to thank TSMC and NSTC for manufacturing and financial support.

References:

- [1] H. Yang et al., "An Empirical Study of Microscaling Formats for Low-Precision LLM Training," *IEEE Symp. on Computer Arithmetic*, pp. 1-8, May 2025. <https://doi.org/10.1109/ARITH64983.2025.00011>
- [2] C. Xiao et al., "Microscaling Vision Transformers on FPGAs," *IEEE Annu. Int. Symp. on Field-Programmable Custom Computing Machines*, p. 275, May 2025. <https://doi.org/10.1109/FCCM62733.2025.00035>
- [3] H. Mun et al., "ASAP: A 28nm Transformer Training Accelerator with Alternating Sparsity and Asymmetrical Microscaling Floating-Point Precision," *IEEE Symp. VLSI Circuits*, pp. 1-3, June 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11075114>
- [4] Y. Jang et al., "Redefining PIM Architecture with Compact and Power-Efficient Microscaling," *Int. Conf. on Electron., Inf., and Communication*, pp. 1-4, Jan. 2025. <https://doi.org/10.1109/ICEIC64972.2025.10879742>
- [5] D.-Q. You et al., "A 22nm 104.5TOPS/W μ -NMC- Δ -IMC Heterogeneous STT-MRAM CIM Macro for Noise-Tolerant Bayesian Neural Networks," *ISSCC*, pp. 250-251, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904540>
- [6] W.-S. Khwa et al., "A 16nm 216kb, 188.4TOPS/W and 133.5TFLOPS/W Microscaling Multi-Mode Gain-Cell CIM Macro Edge-AI Devices," *ISSCC*, pp. 252-253, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904606>
- [7] X. Wang et al., "A 28nm 17.83-to-62.84TFLOPS/W Broadcast-Alignment Floating-Point CIM Macro with Non-Two's-Complement MAC for CNNs and Transformers," *ISSCC*, pp. 254-255, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904738>
- [8] Z. Yue et al., "A 51.6TFLOPS/W Full-Datapath CIM Macro Approaching Sparsity Bound and <2-30 Loss for Compound AI," *ISSCC*, pp. 256-257, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904702>
- [9] Y. Yuan et al., "A 28nm 192.3TFLOPS/W Accurate/Approximate Dual-Mode-Transpose Digital 6T-SRAM CIM Macro for Floating-Point Edge Training and Inference," *ISSCC*, pp. 258-259, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904659>
- [10] X. Chen et al., "A 28nm 64kb Bit-Rotated Hybrid-CIM Macro with an Embedded Sign-Bit-Processing Array and a Multi-Bit-Fusion Dual-Granularity Cooperative Quantizer," *ISSCC*, pp. 260-261, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904646>
- [11] Y. Wang et al., "A 28nm 83.23TFLOPS/W POSIT-Based Compute-in-Memory Macro for High-Accuracy AI Applications," *ISSCC*, pp. 566-567, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454567>
- [12] W.-S. Khwa et al., "A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices," *ISSCC*, pp. 568-569, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454447>
- [13] A. Guo et al., "A 22nm 64kb Lightning-Like Hybrid Computing-in-Memory Macro with a Compressed Adder Tree and Analog-Storage Quantizers for Transformer and CNNs," *ISSCC*, pp. 570-571, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454278>

- [14] H. Fujiwara et al., "A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² Fully-Digital Compute-in-Memory Macro Supporting INT12 $\times$ INT12 with a Parallel-MAC Architecture and Foundry 6T-SRAM Bit Cell," *ISSCC*, pp. 572-573, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454556>
- [15] K. Yoshioka, "A 818-4094TOPS/W Capacitor-Reconfigured CIM Macro for Unified Acceleration of CNNs and Transformers," *ISSCC*, pp. 574-575, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454489>
- [16] Y. Yuan et al., "A 28nm 72.12TFLOPS/W Hybrid-Domain Outer-Product Based Floating-Point SRAM Computing-in-Memory Macro with Logarithm Bit-Width Residual ADC," *ISSCC*, pp. 576-576, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454313>
- [17] Y. He et al., "A 28nm 2.4Mb/mm² 6.9-16.3TOPS/mm² eDRAM-LUT-Based Digital Computing-in-Memory Macro with In-Memory Encoding and Refreshing," *ISSCC*, pp. 578-579, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454323>
- [18] T.-H. Wen et al., "A 22nm 16Mb Floating-Point ReRAM Compute-in-Memory Macro with 31.2TFLOPS/W for AI Edge Devices," *ISSCC*, pp. 580-581, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454468>
- [19] S. Um et al., "Dial: An Energy-Efficient DRAM In-Memory Computing Accelerator with Compact Partial Product LUT and Twisted Differential ADC," *IEEE Symp. VLSI Circuits*, pp. 1-3, June 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11075215>
- [20] H.-H. Hsu et al., "A 22nm 41.8TFLOPS/W AI-Edge Transformer/CNN Nonvolatile-Processor Using QKV-Softmax-Layer-Fused Hybrid ReRAM-CIM and Concurrent-Transpose/Non-Transpose SRAM-CIM," *IEEE Symp. VLSI Circuits*, pp. 1-3, June 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11074985>
- [21] Z. Wu et al., "CELLA: A 28nm Compute-Memory Co-Optimized Real-Time Digital CIM-Based Edge LLM Accelerator with 1.78ms-Response in Prefill and 31.32 token/s in Decoding," *IEEE Symp. VLSI Circuits*, pp. 1-3, June 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11075101>
- [22] M.-E. Shih et al., "NVE: A 3nm 23.2TOPS/W 12b-Digital-CIM-Based Neural Engine for High-Resolution Visual-Quality Enhancement on Smart Devices," *ISSCC*, pp. 360-361, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454482>
- [23] H. An et al., "An 8.09TOPS/W Neural Engine Leveraging Bit-Sparsified Sign-Magnitude Multiplications and Dual Adder Trees," *ISSCC*, pp. 422-423, Feb. 2023. <https://doi.org/10.1109/ISSCC42615.2023.10067269>
- [24] M. Li et al., "A 19.7 TFLOPS/W Multiply-Less Logarithmic Floating-Point CIM Architecture with Error-Reduced Compensated Approximate Adder," *ISCAS*, pp. 1-5, May 2024. <https://doi.org/10.1109/ISCAS58744.2024.10558433>
- [25] Z. Li et al., "Fourier Neural Operator for Parametric Partial Differential Equations," *arXiv preprint*, pp. 1-16, May 2021. <https://doi.org/10.48550/arXiv.2010.08895>
- [26] J. Zhao et al., "LNS-Madam: Low-Precision Training in Logarithmic Number System Using Multiplicative Weight Update," *IEEE Transactions on Computers*, vol. 71, no. 12, pp. 3179-3190, Dec. 2022. <https://doi.org/10.1109/TC.2022.3202747>
- [27] S. Vogel et al., "Efficient Hardware Acceleration of CNNs Using Logarithmic Data Representation with Arbitrary Log-Base," *JCAD*, pp. 1-8, Nov. 2018. <https://doi.org/10.1145/3240765.3240803>
- [28] P. Agghi et al., "Bridging the Gap Between LLMs and LNS with Dynamic Data Format and Architecture Codesign," *MICRO*, pp. 1617-1631, Nov. 2024. <https://doi.org/10.1109/MICRO61859.2024.00118>
- [29] B. Crafton et al., "Finding the Pareto Frontier of Low-Precision Data Formats and MAC Architecture for LLM Inference," *DAC*, pp. 1-7, June 2025. <https://doi.org/10.1109/DAC63849.2025.11132989>
- [30] M.M. Mano and M.D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, p. 51, 2017. Accessed on Dec. 8, 2025. <https://www.pearson.com/en-us/subject-catalog/p/digital-design-with-an-introduction-to-the-verilog-hdl-vhdl-and-systemverilog/P200000003241/9780137501984>

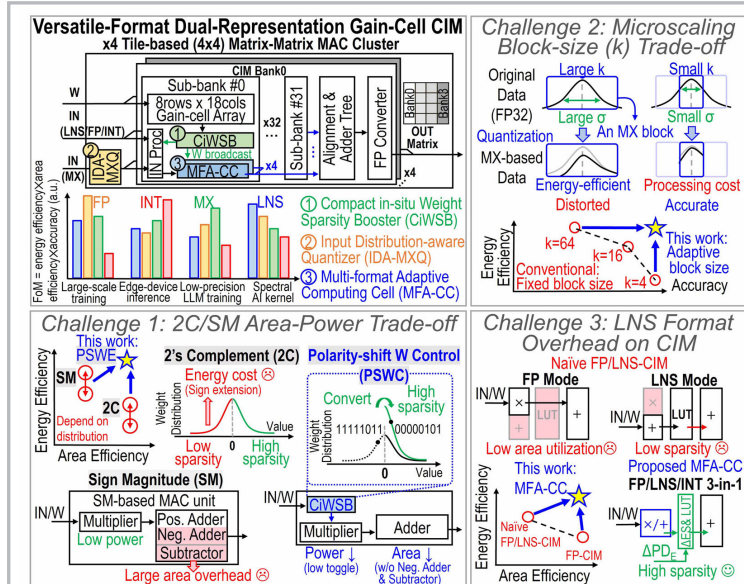


Figure 30.5.1: The proposed VF-DR GC-CIM and challenges of designing a general purpose CIM macro.

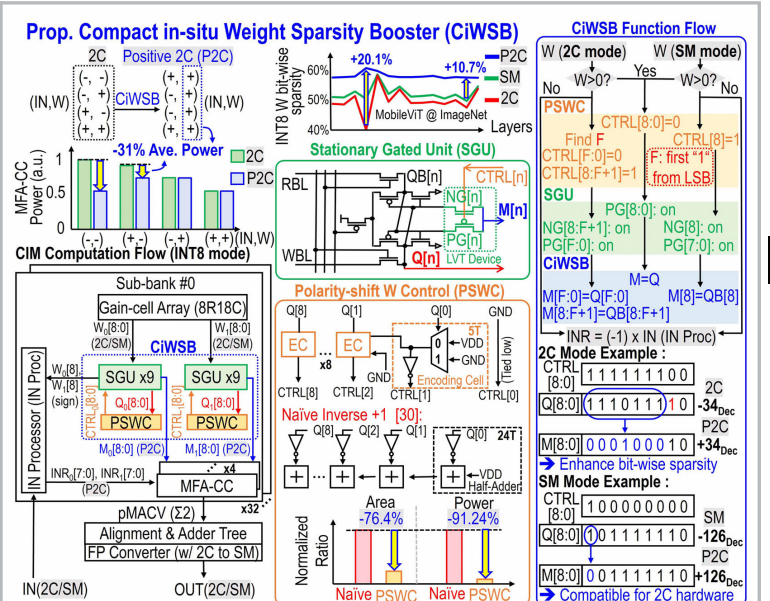


Figure 30.5.2: Proposed CiWSB with SGU, PSWC, and CiWSB function flow.

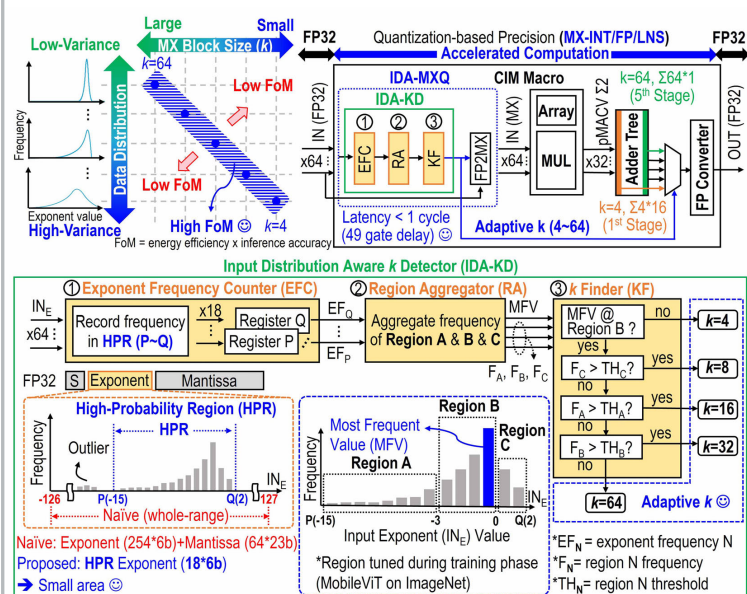


Figure 30.5.3: Proposed IDA-MXQ structure and its operation.

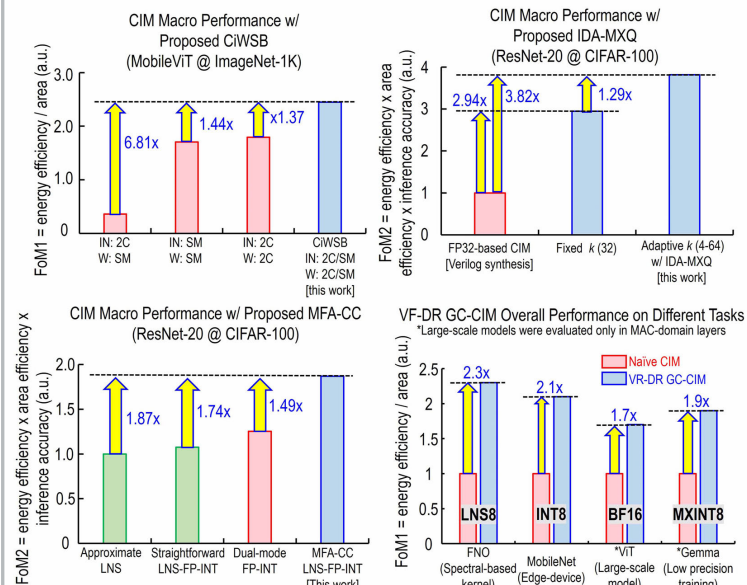


Figure 30.5.5: Simulated performance of proposed schemes.

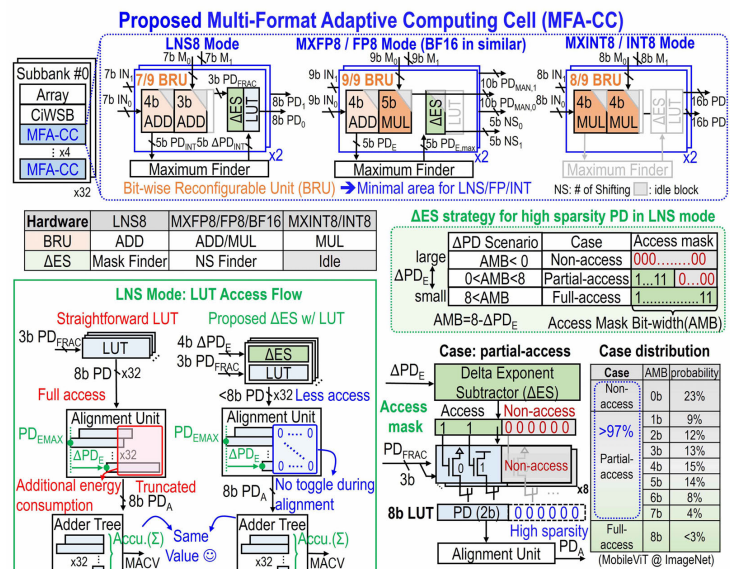


Figure 30.5.4: Proposed MFA-CC structure and operation for versatile-format MAC support.

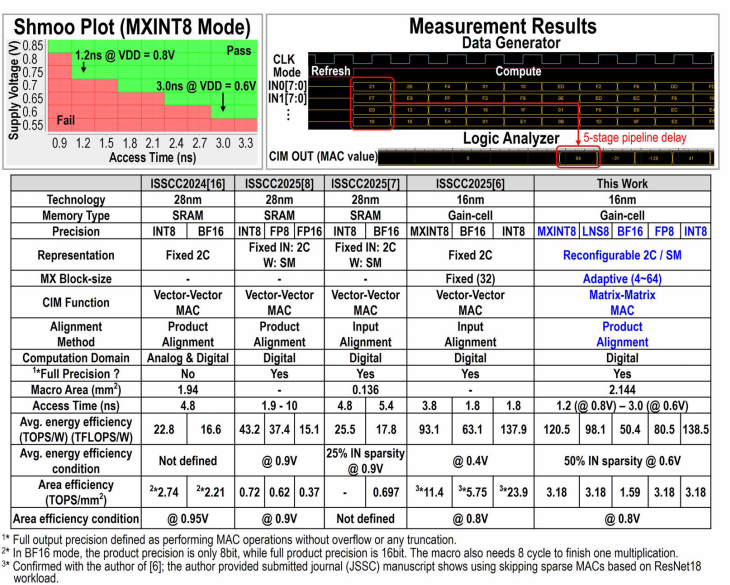


Figure 30.5.6: Measurement results and comparison table.

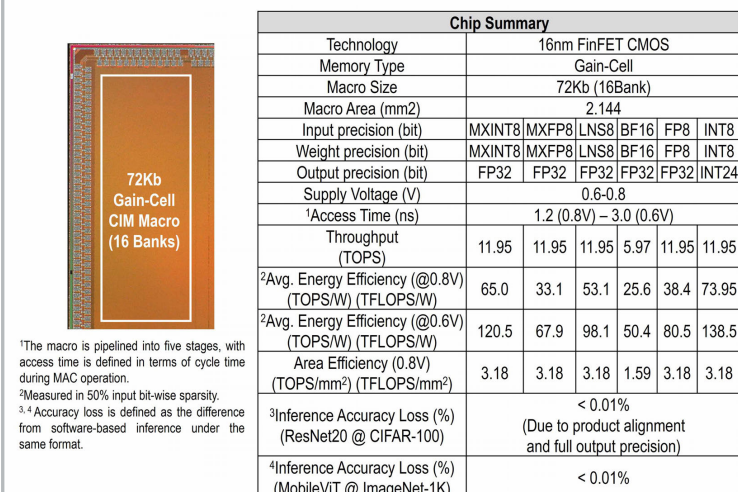


Figure 30.5.7: Die micrograph and chip performance summary table.